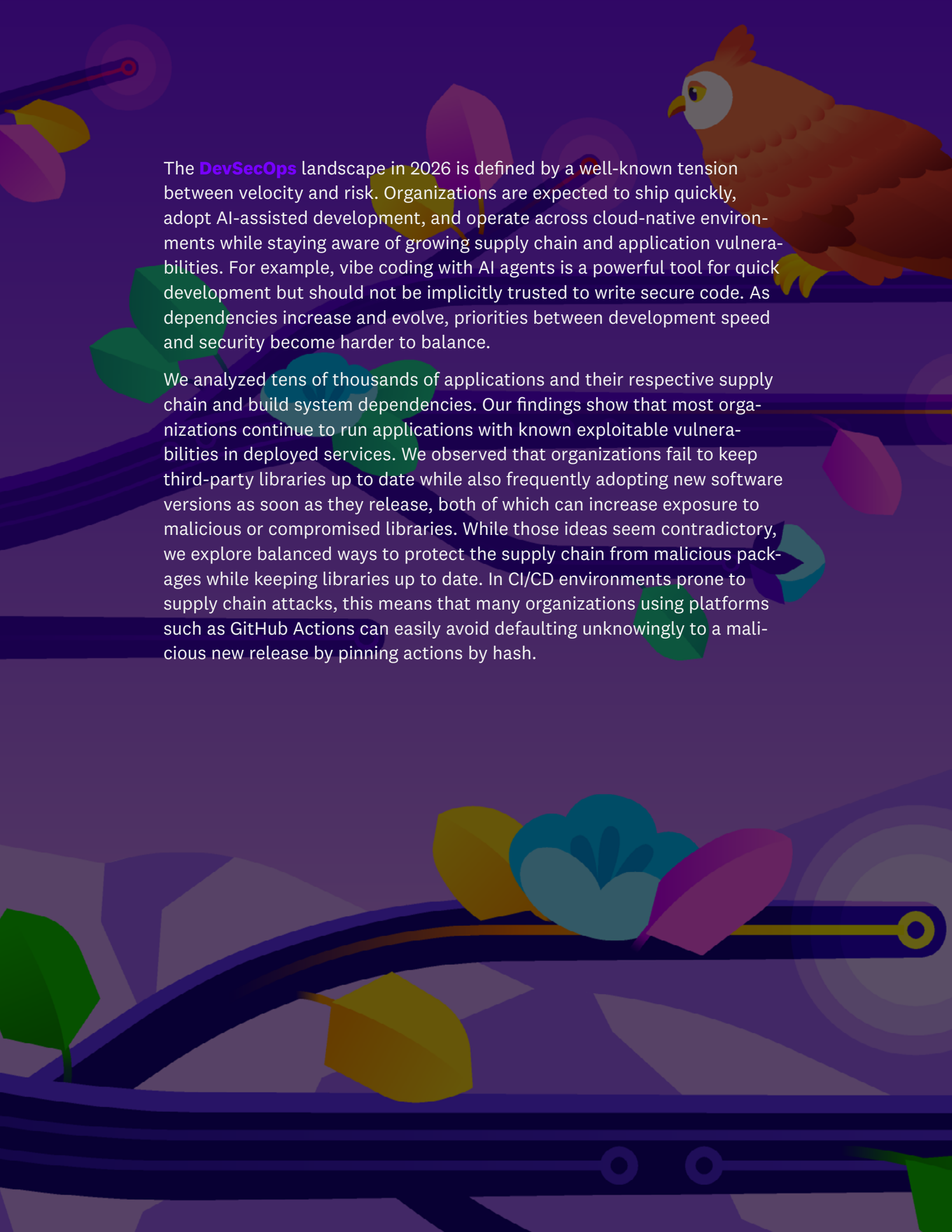


STATE OF DevSecOps





The **DevSecOps** landscape in 2026 is defined by a well-known tension between velocity and risk. Organizations are expected to ship quickly, adopt AI-assisted development, and operate across cloud-native environments while staying aware of growing supply chain and application vulnerabilities. For example, vibe coding with AI agents is a powerful tool for quick development but should not be implicitly trusted to write secure code. As dependencies increase and evolve, priorities between development speed and security become harder to balance.

We analyzed tens of thousands of applications and their respective supply chain and build system dependencies. Our findings show that most organizations continue to run applications with known exploitable vulnerabilities in deployed services. We observed that organizations fail to keep third-party libraries up to date while also frequently adopting new software versions as soon as they release, both of which can increase exposure to malicious or compromised libraries. While those ideas seem contradictory, we explore balanced ways to protect the supply chain from malicious packages while keeping libraries up to date. In CI/CD environments prone to supply chain attacks, this means that many organizations using platforms such as GitHub Actions can easily avoid defaulting unknowingly to a malicious new release by pinning actions by hash.

FACT 1

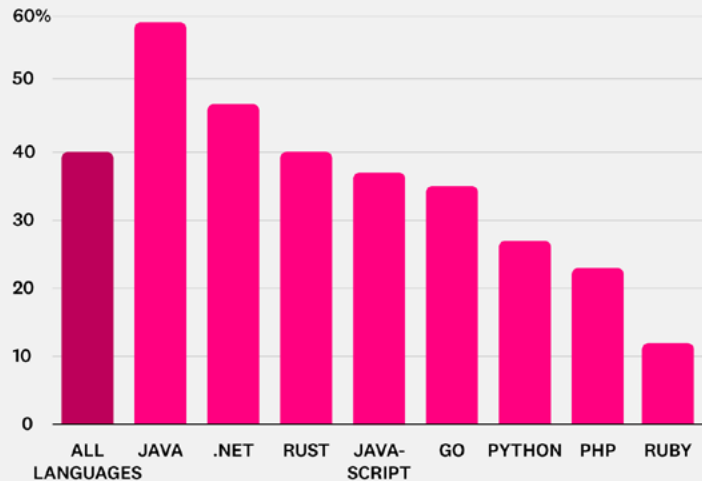
Nearly all organizations have known exploitable vulnerabilities in deployed services

Eighty-seven percent of organizations have at least **one exploitable vulnerability**, affecting **40% of all services**. These vulnerabilities remain most prevalent in Java services at 59%, with .NET at 47% and Rust at 40%. Where vulnerabilities are known to be actively exploited, there's an increased chance that vulnerable services will be compromised.

TWO OUT OF FIVE SERVICES HAVE AN EXPLOITABLE VULNERABILITY

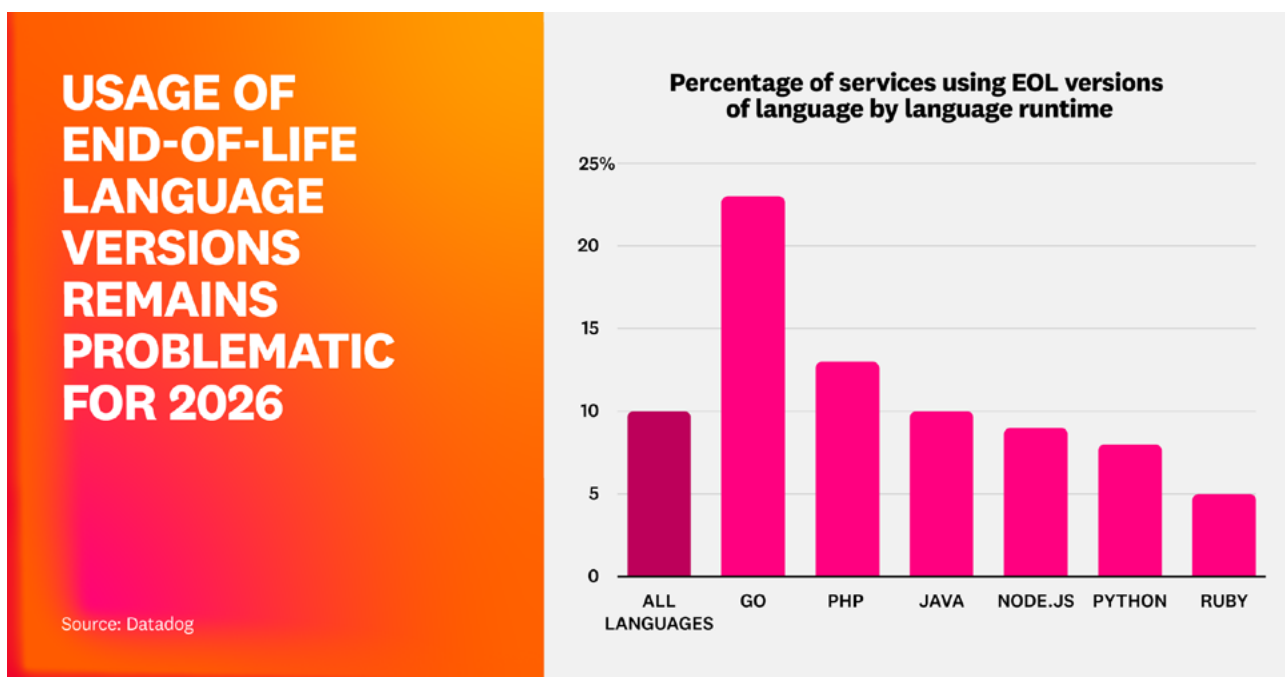
Source: Datadog

Percentage of services with exploitable vulnerabilities by language



Keeping languages and runtime environments up to date is a primary way to prevent exploitable vulnerabilities in applications and services, and it also makes it more likely that dependencies will stay up to date. Once an application falls several versions behind, upgrading to the latest release requires significantly more time, money, and effort, which often causes work to be deferred and ultimately leaves the application running on an end-of-life (EOL) version.

Globally, we found that 10% of services are based on at least **one EOL version** of a language or runtime environment. Java is in the middle of the pack at 10% of services using an EOL version, with Go at 23% and PHP at 13%.

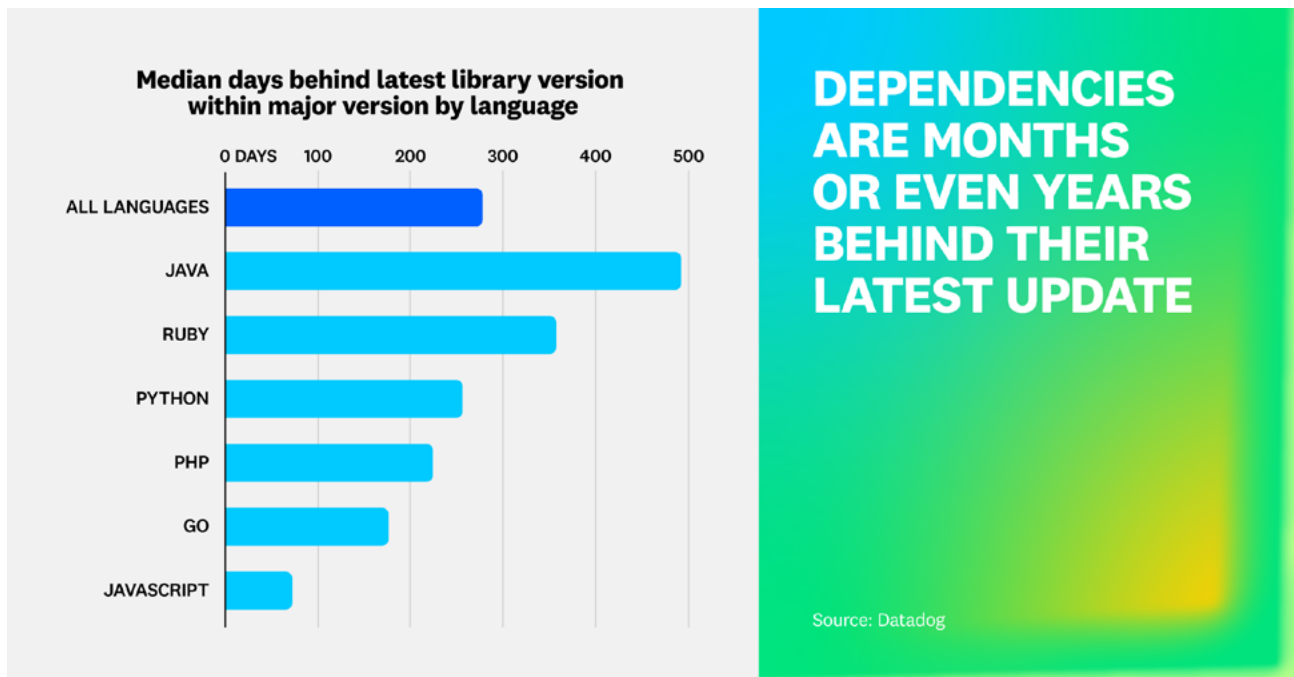


Digging deeper, we found that the aggressiveness of a language's EOL approach may be an influence. For instance, Go takes an aggressive approach with its EOL policy, which is likely causing the spike compared to other languages; a Go version is supported for one year, while PHP versions are supported for four years. When an organization is using an EOL version of a language or runtime environment, 50% of the services in that language have an exploitable vulnerability compared to 7% when there are no EOL versions used.

FACT 2

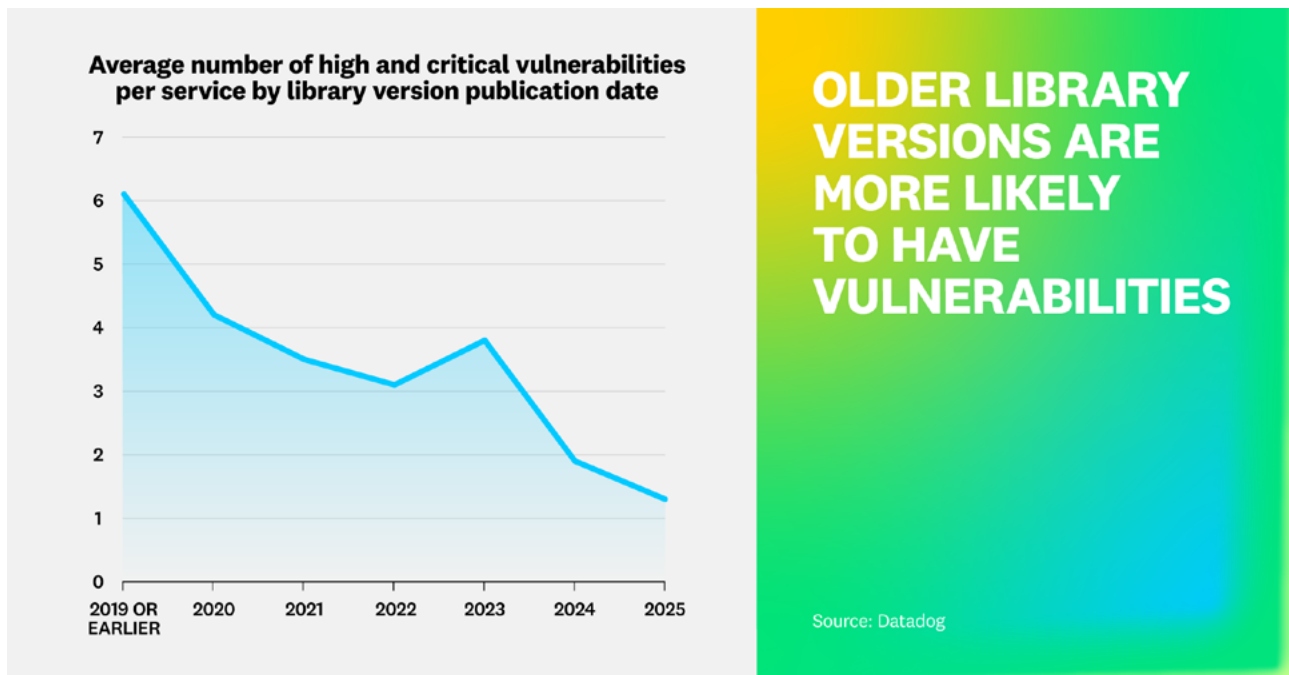
Keeping libraries up to date continues to be a challenge for developers

Modern applications rely on many third-party dependencies, each with its own update cadence and breaking-change risks. Our research found that the median dependency is **278 days behind** its latest major version, compared to 215 days behind last year. Java and Ruby were the two languages above the median at 492 days behind and 357 days behind, respectively.



Dependencies in services that are deployed less than once per month are 70% more outdated than those deployed daily, with a median of 295 days compared to 172 days behind their latest version update. This highlights how deployment frequency is both a productivity metric and a meaningful security control. By evaluating library upgrades with [DevOps Research and Assessment \(DORA\)](#) metrics, we see that greater development velocity and stability correlate with a stronger overall security posture than services with lower DORA metrics.

As mentioned in Fact 1, up-to-date services and dependencies are less likely to have vulnerabilities. This year, we looked at the age of dependencies in relation to the number of vulnerabilities and found that newer libraries have a lower number of discovered vulnerabilities. Broken down by service, libraries published in 2025 have, on average, 1.3 vulnerabilities, compared to 1.9 in 2024 and 3.8 in 2023. The spike in 2023 can largely be attributed to six Spring Framework and Spring Boot CVEs affecting Java services. The prevalence of Java vulnerabilities aligns with our earlier observation that Java dependencies are typically the most outdated.



While our scope for this year's report included only services using Datadog Software Composition Analysis, there was little change year over year in the percentage of services using libraries that are not actively maintained and contain known vulnerabilities. This suggests that companies are still not prioritizing them, potentially due to feature velocity and the growing backlog of vulnerabilities. Security vulnerabilities, deprecations, and supply chain issues can emerge quickly and stack up, making it difficult for teams to track which updates are urgent.

Library vulnerabilities are concerning because they can be discovered in any version at any time. That's why regularly updating libraries often reduces the likelihood of breaking changes and makes future updates easier to successfully complete. Regression testing is key when updating any library, especially when making a big version jump. To start, increasing testing coverage of an application can improve confidence that there were not breaking changes or could be used to identify where an issue occurred.

FACT 3**Updating within a day of release comes with its own risks**

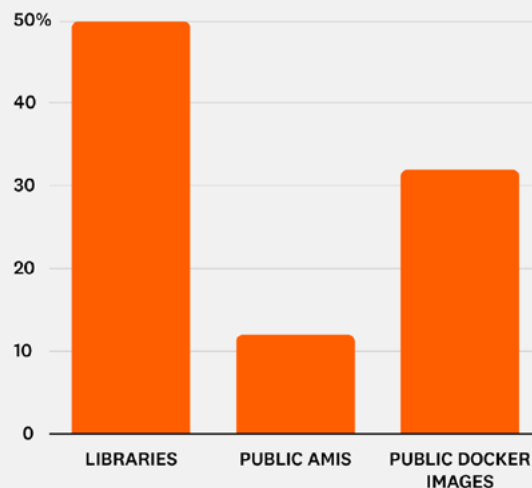
Automatically updating software to a new release may seem like an easy way to keep libraries up to date and avoid the risks discussed in Facts 1 and 2. However, when factoring in supply chain compromises, updating to a new version **within a day of release** can have a negative impact on the overall security of an application due to the potential to unknowingly install malicious software.

To better identify this risk, our research shows that **50% of organizations** are using third-party libraries within a day of release, along with **12% using public AMLs** and **32% using public Docker images** less than one day after release. Despite lower rates of use within a day of release, AMLs and Docker images pose a higher potential impact if compromised due to their broad system-level access when compared to libraries.

HALF OF ALL ORGANIZATIONS USE DAY-OF-RELEASE LIBRARIES, INCREASING THEIR RISK OF USING A MALICIOUS VERSION

Source: Datadog

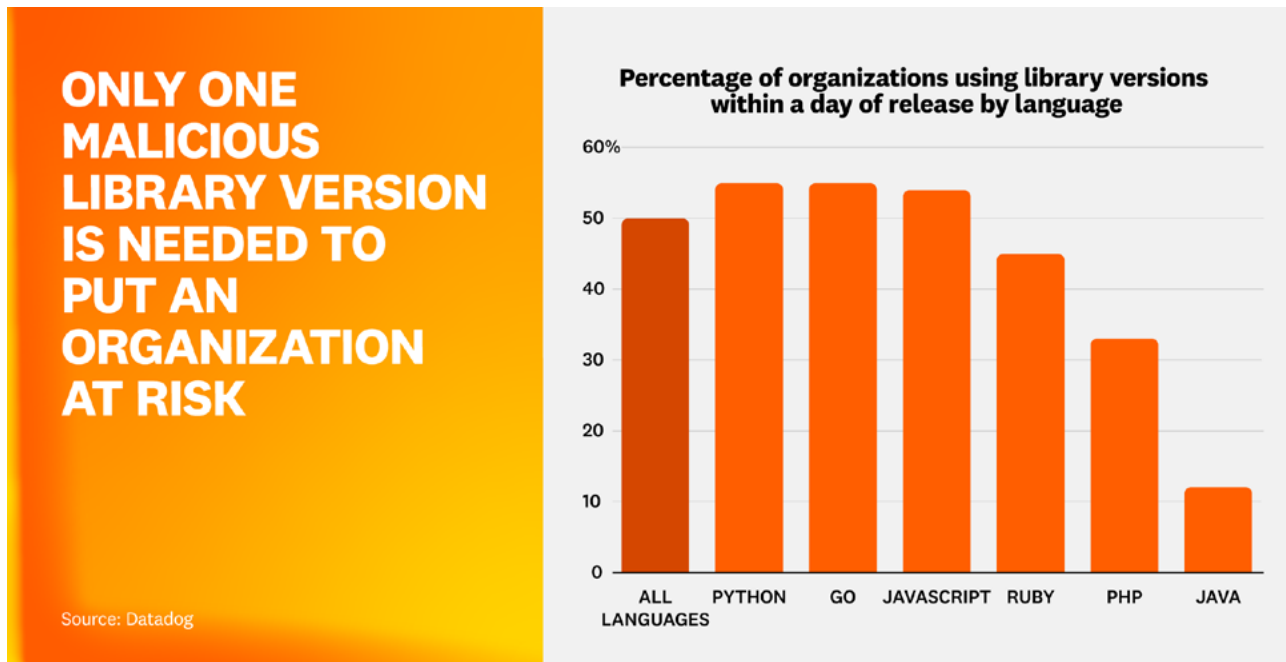
Percentage of organizations using third-party software within a day of release by language

**Libraries**

Over the past year, we have seen the spread of several large-scale supply chain attacks, most notably [s1ngularity](#) and [both Shai-Hulud attacks](#), due in part to organizations using malicious versions of libraries as soon as they're released. To keep dependencies safe from malware, we recommend pinning dependency versions to a full-length commit SHA. If that's not possible, Yarn and pnpm—two alternative JavaScript package managers—have a new configuration for minimum release age to allow time for new versions to be used by others. GitHub's [Dependabot](#) has the ability to add a “cooldown” time for the same reason. This buffer method keeps packages up to date but gives a set time before installation to reduce the likelihood of malicious software. A cooldown of one week could prevent a majority of malicious dependencies from taking root.

Focusing even further, **54% of organizations** using JavaScript and **55% of organizations** using Python used a new library version less than one day after release. We noted that

1.6% of organizations using npm have used at least one malicious dependency in the past year. This is significant at npm's scale; it represents potentially tens of thousands of organizations that executed malicious code, demonstrating that supply chain attacks are an operational reality if proper security measures are not in place.



Public AMIs

In early 2025, Datadog released research on a [cloud image name confusion attack](#) affecting AWS Amazon Machine Images (AMIs). We found that blindly retrieving the most recently created AMI without specifying an owner could lead to malicious AMIs unknowingly being used. Investigating this further, 12% of organizations have used at least one AMI within one day after its creation.

In response to this research, AWS created a new [“Allowed AMIs”](#) feature that can help prevent these attacks, which is recommended if you use a community AMI. If you're looking to see who in your organization has used unverified community AMIs, [whoAMI-scanner](#) is an open source tool that will create a report for you. [Investigator.cloud](#) is another useful resource to explore public AMIs and their lineage.

Docker images

Libraries and AMIs are not the only software that is used immediately after release. In fact, **32% of organizations** have used a public Docker image within one day of its creation, which is also a security risk. Docker containers are not a new attack vector, but the recent research surrounding library and AMI attacks has overshadowed the malicious container images seen in the past.

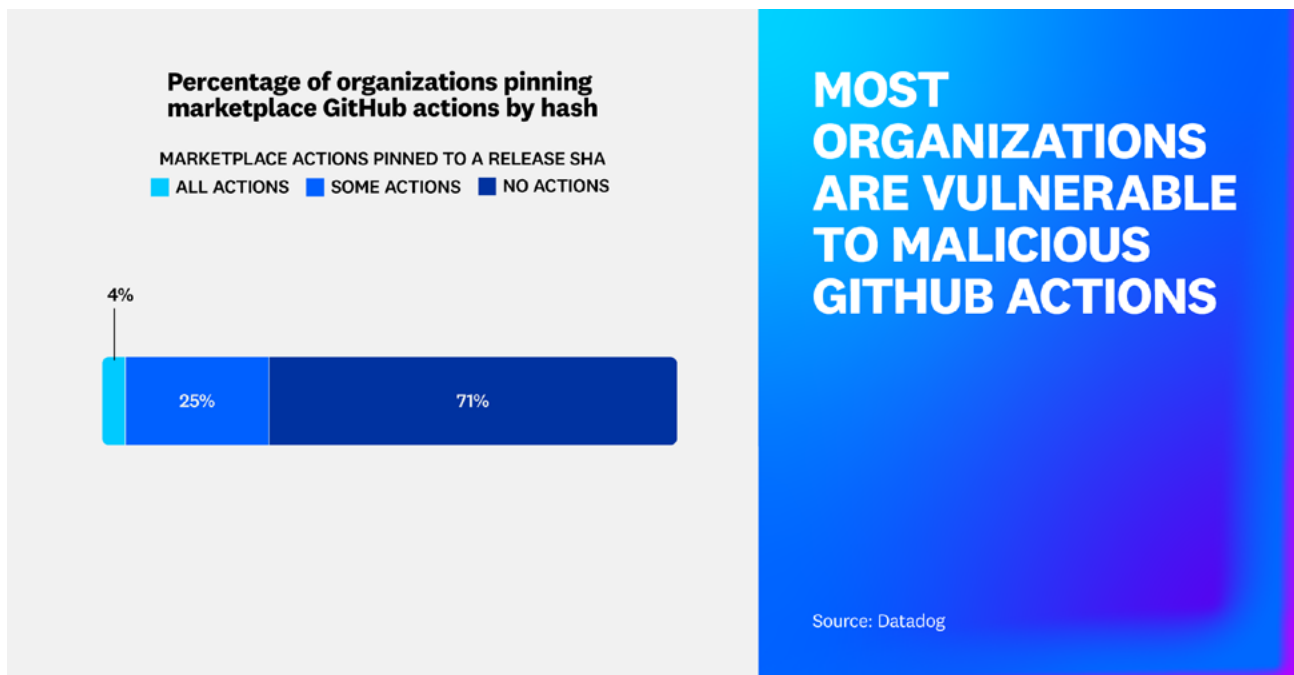
Whenever possible, ensure that container images are coming from trusted first-party sources, such as using the “Trusted Content” tags in Docker Hub (e.g., [Docker Official Images](#), [Verified Creators](#), and [Sponsored OSS](#)) to reduce the risk of using a malicious image. If you aren't able to use an image from a first-party trusted source, consider allowing a cooldown period to give researchers time to investigate along with doing your own analysis. Nothing is foolproof, but these options can help reduce the risk associated with using a public image.

FACT 4

GitHub Actions is left vulnerable to supply chain attacks

This past year, we have seen an increase in large-scale supply chain attacks that used GitHub Actions in a variety of ways to gain initial access to repositories and exfiltrate data that affects both custom and marketplace GitHub actions. The `changed-files` action by `tj-actions` was compromised, and attackers inserted a malicious payload that caused affected public repositories to write secrets to workflow logs.

For this report, we are defining marketplace actions as publicly available actions on [GitHub's Marketplace](#). While GitHub Actions vulnerabilities have remained relatively consistent, we've found that every organization using GitHub Actions uses at least one marketplace action, but only 4% of organizations pin the hash for all marketplace actions. Additionally, 71% of organizations never pin the hash for any of their actions. This is risky because GitHub states that [pinning an action to a full-length commit SHA](#) is the only way to prevent automatic updates, as discussed in Fact 3.



Digging deeper, we found that **80% of organizations** use at least one marketplace action not managed by GitHub and not pinned to a commit hash. This may reflect limited awareness of the guidance and the added inconvenience of needing to update the YAML file.

Two percent of organizations that use GitHub Actions are running at least one action that has been compromised in the past and is not pinned to a commit hash. With the growing frequency of compromised actions, this figure is expected to increase year over year unless adoption of commit pinning actions improves.

“GitHub Actions workflows routinely handle production credentials, generate release builds, and execute third-party code in highly privileged contexts, yet most organizations don’t secure them with the same rigor they apply to production infrastructure. The major GitHub Actions compromises of 2025 made clear that attackers are systematically targeting this gap. Organizations need a first-principles approach to CI/CD security: Understanding the blast radius of every workflow, monitoring runtime behavior, and treating build infrastructure as a high-value asset. That foundational thinking is what enabled early detection of attacks like tj-actions, Shai-Hulud, and s1ngularity, and it’s what the industry needs to adopt broadly in 2026.”

Ashish Kurmi
Co-Founder & CTO at StepSecurity

FACT 5

Most vulnerabilities should not page a human

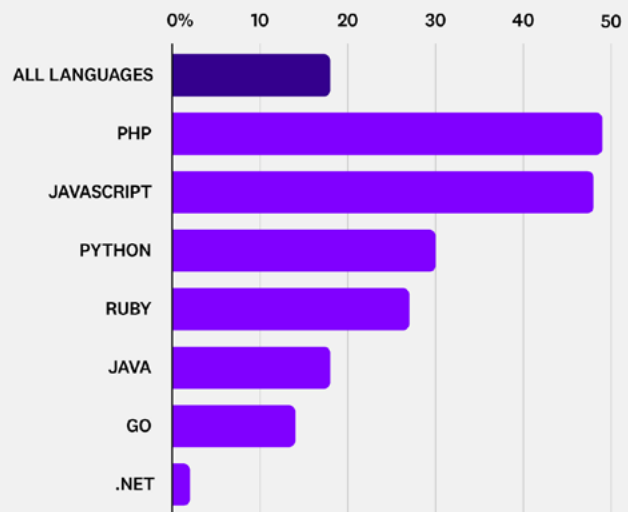
Critical vulnerabilities should be quickly fixed, but only when they are truly critical. Vulnerability severity—typically measured by its [CVSS base score](#)—is the starting point but can be adjusted when taking runtime context and CVE context into account. Our research shows that only **18% of critical dependency vulnerabilities stay critical** after adjusting the severity score, the same as last year. Truly critical vulnerabilities are reduced by more than 80%, substantially decreasing alert volume.

**LESS THAN 1 IN 5
CRITICAL
VULNERABILITIES
ARE WORTH
PRIORITIZING**

* After incorporating runtime context and exploitability signals

Source: Datadog

Percentage of critical vulnerabilities remaining critical*



This year, we wanted to dig further and understand how this differs across languages. We found that 98% of .NET dependency vulnerabilities are downgraded from critical. After further investigation, the average Exploit Prediction Scoring System (EPSS) score for a .NET vulnerability is much lower than that of other languages, likely due in part to the built-in security features, like Code Access Security, that reduce exploit probability. On the higher end of the spectrum, 49% of PHP dependency vulnerabilities remain critical. This may be due to the high public exploit availability and high exploitation probability of many PHP dependency vulnerabilities.

The Datadog severity score that is used to adjust the initial CVSS score is based on four factors. For the runtime context, it considers whether the vulnerability is in production and whether the affected service is under active attack. The two additional factors are derived from CVE context: the availability of an exploit and the likelihood of exploitation.

Over-prioritizing every critical vulnerability can overwhelm teams, creating alert fatigue and diverting attention from vulnerabilities that pose genuine business risk. Better prioritization allows teams to fix what matters most, which saves developers time and ultimately reduces costs.

We did notice that the **average** number of high or critical vulnerabilities per application that has at least one SCA vulnerability **decreased to eight** from 13.5 in 2025. This downward trend will be a key indicator for whether security prioritization is improving over the coming years.

Key learnings from the 2026 State of DevSecOps study

We recently released the 2026 State of DevSecOps study, in which we analyzed tens of thousands of applications and their respective supply chain and build system dependencies. Our research revealed trends in security posture and best practices across the software development life cycle. In particular, we found that:

1. Nearly all organizations have known exploitable vulnerabilities in deployed services
2. Keeping libraries up to date continues to be a challenge for developers
3. Updating within a day of release comes with its own risks
4. GitHub Actions is left vulnerable to supply chain attacks
5. Most vulnerabilities should not page a human

In this post, we outline several best practices that organizations should implement based on these findings, including to:

- Better secure GitHub Actions
- Use DORA metrics to drive measurable security improvements
- Upgrade software quickly, but not too quickly
- Use application and vulnerability context for better prioritization

In addition, we explain how you can use Datadog Code Security and Cloud SIEM within the Datadog platform to improve your security posture and defend against threats.

Better secure GitHub Actions

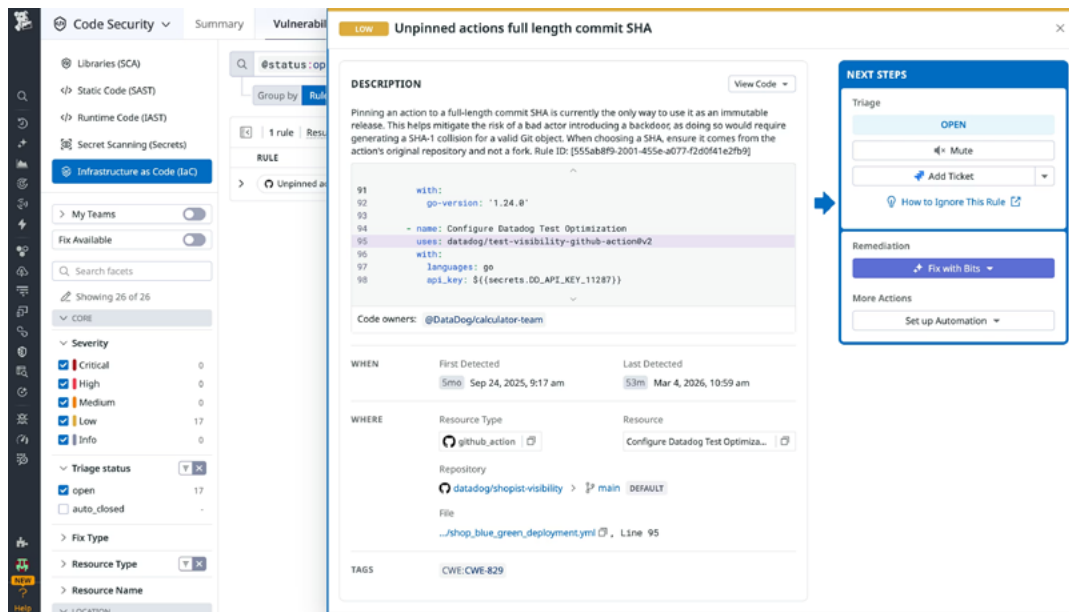
CI/CD pipelines are the backbone of modern engineering, but they are also a prime target for attackers. Our research highlights that GitHub Actions is particularly vulnerable to supply chain attacks if not properly secured. Relying on tags such as @v1 or @latest for third-party actions means an attacker who compromises the repository can inject malicious code directly into your build pipeline. GitHub **strongly recommends** pinning actions to a specific, immutable commit SHA.

```
1. # the secure way: pinning to a commit SHA
2.uses: thollander/actions-comment-pull-request@b07c7f86be
    67002023e6cb13f57df3f21cdd3411
```

```
1. # the insecure way: pinning to a tag
2.uses: thollander/actions-comment-pull-request@v2
```

Because this can be tedious to update manually, GitHub's [Dependabot](#) has the capability to create pull requests that will update the hash to the latest version on a regular cadence.

Datadog Infrastructure as Code (IaC) Security provides comprehensive visibility into your CI/CD configurations. Datadog includes [out-of-the-box IaC rules for GitHub Actions](#) that automatically scan your workflow files and flag risky configurations, such as failing to pin actions to a specific commit hash, so you can secure your pipelines before code ever reaches production.

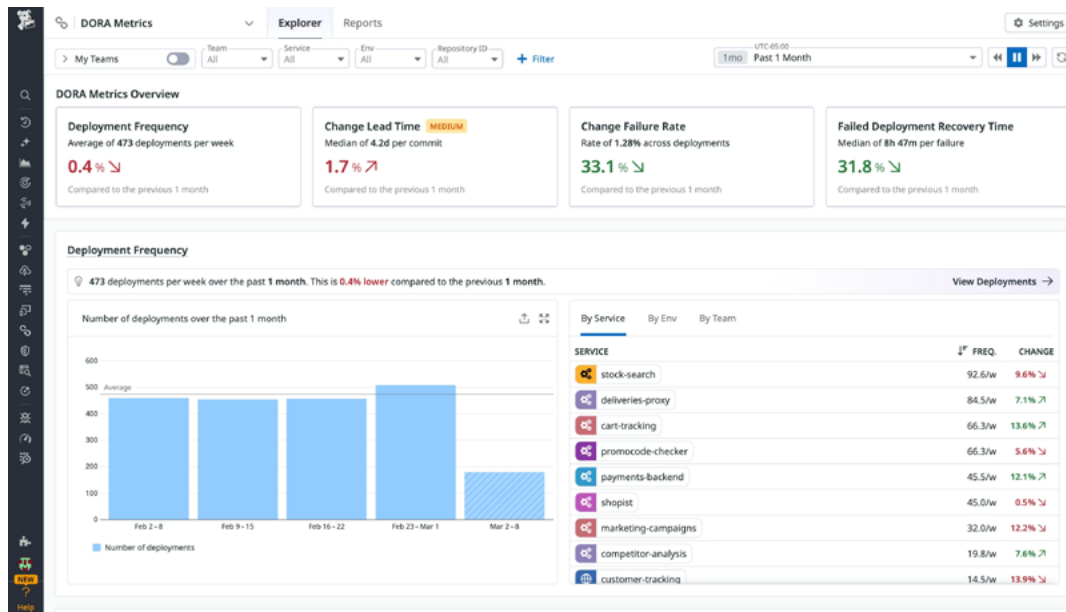


Use DORA metrics to drive measurable security improvements

[DevOps Research and Assessment \(DORA\)](#) metrics are traditionally used to indicate the velocity and stability of software delivery, but we found that improvements in software delivery performance are decidedly correlated with stronger security posture. High-performing teams deploy frequently and with shorter lead times without increasing failures, indicating a delivery process that is both efficient and resilient. Research and industry practice show that teams with higher DORA metrics not only ship code faster but also recover from failures more quickly. This creates smaller windows of exposure for vulnerabilities and faster feedback loops for patching and upgrades.

When applied to dependency management, this means that teams with better DORA metrics can integrate library upgrades continuously, reducing the backlog of outdated or vulnerable packages and improving the overall security posture relative to services with lower DORA scores. By aligning deployment velocity with security goals, organizations benefit from a cycle where frequent, stable deployments support both rapid innovation and reduce risk.

To help teams act on insights surfaced through DORA metrics, Datadog provides an integrated suite of tools spanning performance monitoring, CI visibility, code quality analysis, and incident response. The [DORA Metrics](#) page centralizes all four key indicators—deployment frequency, change lead time, change failure rate, and time to restore service—in a single view, using clear visualizations to make performance trends and regressions easy to identify. Teams can also segment and filter metrics by attributes such as service, environment, or team, allowing them to tailor analyses to specific workflows and focus improvement efforts where they will have the greatest impact.



Upgrade software quickly, but not too quickly

While keeping software up to date is crucial, blind automatic upgrades come with their own set of risks. We found this was true especially for AMIs and libraries.

Libraries

Recent supply chain incidents demonstrate how quickly malicious dependencies can inflict widespread damage once introduced into the software ecosystem. Campaigns such as [s1ngularity](#) and both [Shai-Huluds](#) used malicious packages published to public registries, where they were unknowingly pulled into downstream projects. Automatic dependency updates and loosely pinned version constraints accelerated the spread, allowing compromised packages to propagate rapidly without manual intervention. What might have begun as a single malicious upload quickly cascaded into thousands of installations. This illustrates how modern development practices, particularly automated upgrades designed to improve velocity, can inadvertently amplify the blast radius of a supply chain attack when adequate guardrails are not in place.

For proactive supply chain defense, use open source CLI tools like [GuardDog](#) and [Supply-Chain Firewall](#). GuardDog helps identify suspicious or malicious behavior in open source packages—such as obfuscation, data exfiltration patterns, or typosquatting indicators—before they are introduced into your codebase. The Supply-Chain Firewall can automatically block known malicious packages from being installed in development environments, preventing compromised dependencies from entering your software life cycle in the first place. The Supply-Chain Firewall is partially powered by Datadog’s open source [malicious software package dataset](#), which is continuously updated by security researchers and

GuardDog, tracking real-world supply chain campaigns. Together, they give organizations both visibility and preventative controls to reduce exposure to malicious dependencies upstream.

While often avoided due to the hassle of updating, pinning packages to a full-length commit SHA is recommended to reduce the risk of introducing malicious dependencies. Alternative safeguards can help create a protective buffer. For example, [Yarn](#) and [pnpm](#) now support configurations that enforce a minimum release age, delaying the installation of newly published package versions until they have been available for a defined period. Similarly, GitHub's Dependabot offers a [cooldown](#) setting that postpones automated updates for the same purpose. This intentional delay preserves the benefits of staying current while reducing the likelihood of pulling in a freshly published malicious package. A cooldown of one week could prevent a majority of malicious dependencies from spreading.

PNPM-WORKSPACE.YAML

```
1. minimumReleaseAge: 10080 # one week in minutes
```

.YARNRC.YML

```
1. npmMinimalAgeGate: "7d"
```

DEPENDABOT.YML

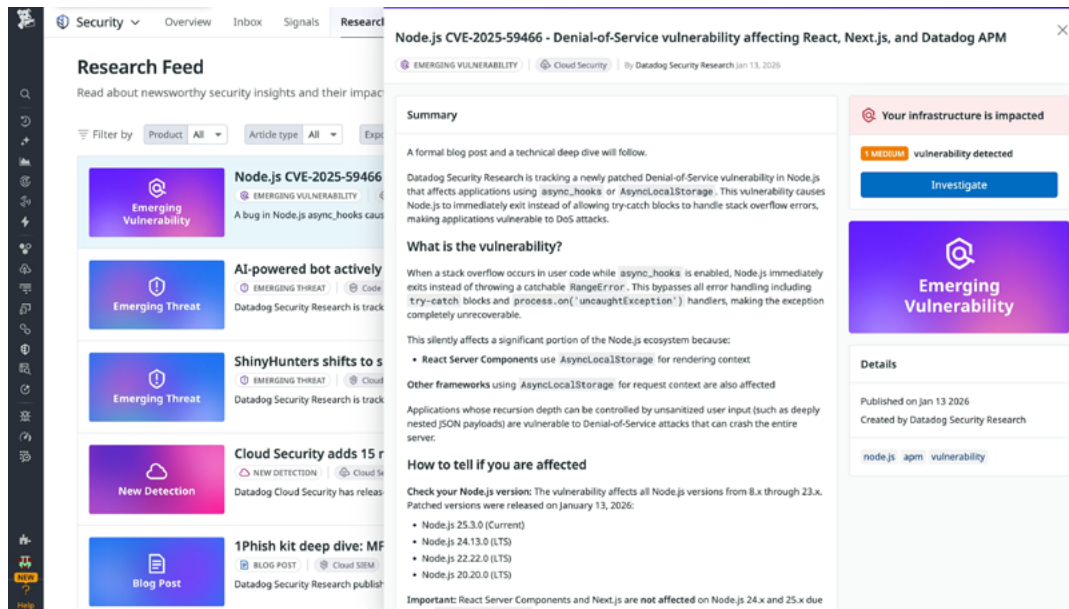
```
1.    cooldown:
2.    default-days: 7
```

[Datadog Software Composition Analysis \(SCA\)](#) helps teams identify known malicious packages before they make it into production by continuously analyzing dependencies across repositories and services. SCA detects vulnerable and suspicious packages using curated threat intelligence and vulnerability databases, enabling teams to surface risks early in development and in deployed applications. Through the Libraries (SCA) tab in Code Security, developers can search across services and directories to identify vulnerable or risky packages and understand where they are introduced.

The screenshot displays the Datadog Code Security interface, specifically the 'Libraries (SCA)' tab. The left sidebar shows navigation options like 'Static Code (SAST)', 'Runtime Code (IAST)', 'Secret Scanning (Secrets)', and 'Infrastructure as Code (IaC)'. The main panel shows a search for 'status: Open' with 1-25 of 7K vulnerabilities. The table lists several critical vulnerabilities, including CVE-2023-46604 (Apache ActiveMQ), CVE-2021-44228 (Remote code injection in Log4j), CVE-2017-5638 (Apache Struts), and CVE-2021-30181 (Code injection in Apache Dubbo). Each entry includes the severity, vulnerability details, and actions like 'Set up Automation', 'Download as CSV', and 'Ticketing'.

SEVERITY	VULNERABILITY	DETECTED IN	TICKETING
CRITICAL 10	Apache ActiveMQ is vulnerable to Remote Code Execution CVE-2023-46604 org.apache.activemq:activemq-client 5.14.5 Reachable High Exploitation Risk Exploit Available	1 Repository 0 Services	1 Ticket
CRITICAL 10	Remote code injection in Log4j CVE-2021-44228 org.apache.logging.log4j:log4j-core 2.1 High Exploitation Risk Exploit Available	1 Repository 0 Services	
CRITICAL 10	Apache Struts vulnerable to remote arbitrary command execution due to improper input validation CVE-2017-5638 org.apache.struts:struts2-core 2.3.1 High Exploitation Risk Exploit Available	1 Repository 0 Services	
CRITICAL 10	Apache Struts vulnerable to remote arbitrary command execution due to improper input validation CVE-2017-5638 org.apache.struts:struts2-core 2.3.4 High Exploitation Risk Exploit Available	1 Repository 0 Services	
CRITICAL 10	Apache Struts vulnerable to remote arbitrary command execution due to improper input validation CVE-2017-5638 org.apache.struts:struts2-core 2.5.1 High Exploitation Risk Exploit Available	1 Repository 0 Services	
CRITICAL 10	Code injection in Apache Dubbo CVE-2021-30181 com.alibaba:dubbo 2.5.10 High Exploitation Risk Exploit Available	1 Repository 0 Services	

In addition, the [Security Research Feed](#) provides timely alerts about emerging supply chain attacks, malicious package campaigns, and newly disclosed threats, along with visibility into whether your environment is affected. Together, these capabilities allow organizations to quickly detect, investigate, and remediate malicious dependencies.



AMIs

Amazon Machine Images (AMIs) are [vulnerable to name-confusion attacks](#), which highlights how a seemingly minor misconfiguration in how AMIs are selected can lead to serious compromise of cloud environments. In this attack, software that retrieves the latest AMI using the AWS `ec2:DescribeImages` API without explicitly specifying a trusted owner can be tricked into launching a malicious AMI published by an attacker with a carefully crafted name. Because anyone can publish an AMI to the public catalog, this name-confusion pattern can result in an attacker's image being selected instead of the intended one, potentially granting remote code execution within the victim's AWS account. This underscores that AMI selection is not just a convenience issue but a genuine supply chain threat vector requiring guardrails and detection.

In response to this research, AWS introduced the [“Allowed AMIs”](#) feature, which helps organizations restrict instance launches to trusted images—a particularly important safeguard when using community AMIs. For teams that want visibility into past usage, the open source [whoAMI-scanner](#) can generate reports identifying where unverified community AMIs have been deployed within an organization. Additionally, [Investigator.cloud](#) provides a way to analyze public AMIs and trace their lineage, offering deeper insight into image provenance and potential risk.

To detect when the most recent AMI is being used without an owner or filter, Datadog Infrastructure as Code (IaC) Security created a [rule to check your Terraform code](#) for this misconfiguration. Additionally, Datadog's Cloud SIEM has a [rule to alert you](#) whenever the `ec2:DescribeImages` endpoint is queried without using owner identifiers and then followed by an `ec2:RunInstances` call by the same ARN.

Signal | Tue, Mar 3, 2026, 12:29:20 pm | 3 minutes ago | [Share](#) | [Explore Rule](#) | [X](#)

MEDIUM **EC2 instance created using risky AMI search pattern - Call to DescribeImages without specifin...**

[attack](#) | [TA0001-Initial-Access](#) | [T1195-Supply-Chain-Compromise](#) | Past month signal count: 3 | Past day signal count: 1

WHAT HAPPENED [Threshold](#)

This signal is generated from a detection method that triggers when a rule query has exceeded a threshold.

CONTEXT

Source	Service	Environment	Other tags
cloudtrail	cloudtrail	Tag not available	scope:... +27

USERS

[Redacted] with a risk score of 5

IPS

66.66.221.16

RULE DETAILS

3 logs matched

query describe_images_with_start_without_owners

for userIdentity.arn:arn:aws:sts:[Redacted]

TRIGGER

Triggers when query describe_images_with_start_without_owners > 0

Then

RULE DETAILS

1 log matched query run_instances

for userIdentity.arn:arn:aws:sts:[Redacted]

TRIGGER

Triggers when query run_instances > 0

LOG ACTIVITY [See More >](#)

NEXT STEPS

Triage

[OPEN](#)

[Assign Signal](#)

[0 Comments](#)

Take action

[Create Case](#)

[Declare Incident](#)

[Edit Suppressions](#)

[Run Workflow](#)

[Investigate Risky Entities](#)

Use application and vulnerability context for better prioritization

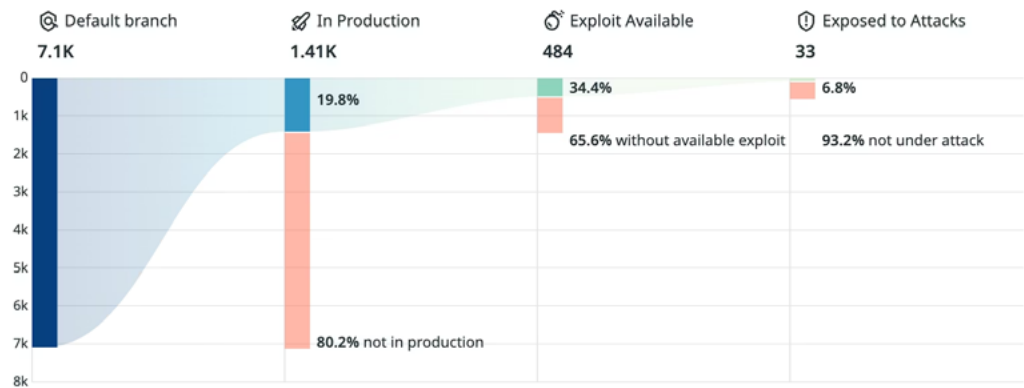
Dependency vulnerabilities are widespread, and the sheer volume of findings across modern codebases can quickly overwhelm even well-resourced security teams. In many organizations, prioritization still relies heavily on [Common Vulnerability Scoring System \(CVSS\) ratings](#). While CVSS provides a useful baseline for estimating theoretical severity, it does not account for how a vulnerability manifests within a specific environment. As a result, teams often treat every “critical” vulnerability as equally urgent, regardless of whether it is exploitable in practice.

Our research shows that only 18% of vulnerabilities initially labeled as critical should remain critical when evaluated with additional context; the remaining 82% can be downgraded based on factors such as runtime exposure, exploitability, and reachability. This gap highlights a fundamental challenge in vulnerability management: Severity alone does not equal risk. Without environmental context, security teams are left chasing high scores rather than meaningful threats.

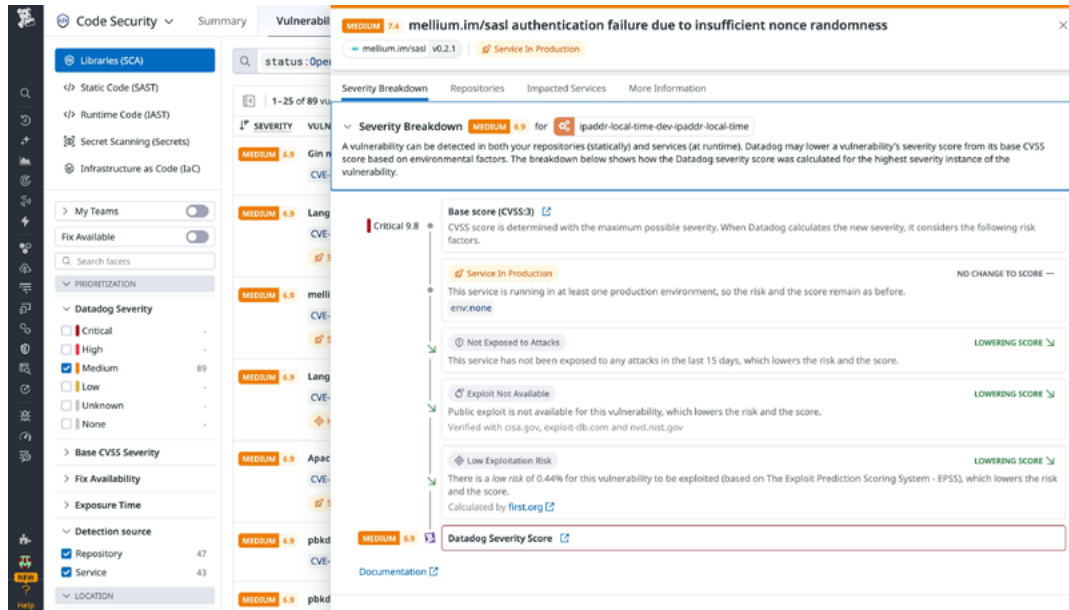
We also found that 87% of organizations have a known exploitable vulnerability in deployed services, which is surprising given that many vulnerabilities are downgraded when no exploit is available. Even as improved severity scoring reduces noise by deprioritizing issues that lack real-world exploitability, a significant subset of vulnerabilities still meets the higher bar of being demonstrably exploitable. Organizations should focus on fixing vulnerabilities that are in production and have an exploit available.

Vulnerabilities from default branch to production

Prioritize fixing vulnerabilities that present the highest risk



Datadog's updated severity scoring model addresses this problem by incorporating runtime context and exploit intelligence into vulnerability prioritization. The Datadog severity score refines traditional CVSS ratings to better reflect the actual risk posed to a given service or application. By accounting for whether a vulnerable dependency is in production, reachable in execution paths, or associated with active exploitation signals, the score provides a more accurate representation of urgency. The result is a clearer remediation roadmap that reduces noise, improves focus, and enables teams to allocate effort where it will have the greatest security impact.



Secure your SDLC with Datadog

To maintain high release velocity without compromising security, organizations need comprehensive visibility across their entire software lifecycle, from vulnerabilities in application code to misconfigurations in cloud infrastructure. They also need the context required to prioritize and remediate meaningful risks. Datadog brings these insights together in the same platform used to monitor system health and performance, eliminating operational silos and enabling teams to collaborate more effectively. By unifying security and observability, organizations can accelerate DevSecOps adoption while driving measurable improvements in security outcomes.

Check out [our documentation to get started](#). If you're not a customer, get started today with a 14-day [free trial](#).

Methodology

Fact 1

For this fact, we analyzed vulnerabilities in third-party libraries of applications across various languages and runtimes (Java, .NET, PHP, Python, Ruby, JavaScript, and Go) that use Datadog Code Security's Software Composition Analysis feature. We sourced known exploited vulnerabilities from the CISA KEV catalog, NIST catalog, ExploitDB, and GitHub Advisories, which we extracted in January 2026. To identify end-of-life versions of languages, we used <https://endoflife.date/>.

Fact 2

For this fact, we collected data on libraries that were active in January 2026. For each dependency, we calculated the number of days since the latest update to the current major version of a library was released. We then computed median lags across ecosystems (e.g., Java, Go). We only looked at libraries that follow Semantic Versioning and excluded public libraries from our analysis.

For frequency of deployment, we obtained the data from the `datadog.service.time_between_deployments` metric, looking at deployments in the past six months. If a service was deployed 100 times or more in the last six months, we counted the service as being deployed daily, as there are 100 business days in six months. If a service was deployed 26 times or more in six months, we counted it as being deployed weekly. If the service was deployed six times or more, we counted it as being deployed monthly, and we categorized services deployed less than six times in six months as being deployed less than monthly.

We analyzed vulnerabilities in third-party libraries called by applications that use Datadog Code Security's Software Composition Analysis.

Fact 3

For this fact, we collected data on libraries that were active in January 2026. We only looked at libraries that follow Semantic Versioning and excluded public libraries from our analysis.

For Docker images and public AMIs, we used Datadog internal product data to identify when an org started using a given Docker image (resp. public AMI) and compared it to its publication date, derived from [Docker Hub](#) and [Investigator.cloud](#).

For malicious dependencies, we looked at organizations and services based on npm ecosystem libraries in 2025. We computed the share of these services and organizations having used [identified malicious versions](#).

Fact 4

For this fact, we analyzed static code vulnerabilities through December 2025 that had the following vulnerability: 'github-actions/unpinned-actions'. We also analyzed our internal CI Visibility product data through January 2026 to understand usage of GitHub Actions. We only focused on GitHub Actions having a standard template (run, post, build).

When analyzing compromised actions, we used the following query:

```
(action_name LIKE any('%tj-actions/changed-files%', '%reviewdog/  
action-setup%', '%tj-actions/eslint-changed-files%'))
```

Fact 5

We analyzed vulnerabilities in third-party libraries called by applications that use Datadog Code Security's Software Composition Analysis feature.

We considered vulnerabilities with a "critical" CVSSv3 base score and, based on the context available, used the following methodology to compute the "temporal" and "environmental" CVSSv3 metrics:

When the service was running in a non-production environment, we adjusted the "modified confidentiality, integrity, and availability impact" to "low."

When the service was not publicly exposed on the internet and the exploit vector was "network," we set the "modified attack vector" to "local."

When the EPSS score was below 1%, we set the "modified attack complexity" to "high."

When a public exploit was available, we set the "exploit code maturity" to "proof of concept" or to "unproven" otherwise.

We then computed the adjusted score based on the CVSS v3.1 methodology and considered the ratio of vulnerabilities whose adjusted score was still "critical."

Licensing

Report: CC BY-ND 4.0

Images: CC BY-ND 4.0



DATADOG

datadog.com